

RAG, Fine-Tuning and Prompt Engineering: Choosing How to Improve an LLM

Know which of the three levers to pull, what each one actually costs, and how to layer them.

For engineers building on top of language models who have to decide where the effort goes · 26 July 2026

The one- or two-page version: what matters, the topics it covers, and every term defined. Read the full lesson for the explanations and worked examples.

[Watch the source video](#)[Read the full lesson](#)[Read the condensed version](#)

What matters

- ✓ Parametric knowledge is fixed at the training cutoff — anything newer or private has to arrive through the prompt.
- ✓ RAG is retrieve, augment, generate: the weights are untouched, only the model's input changes.
- ✓ Retrieval matches meaning rather than words, because both the query and the corpus are converted to vectors and compared geometrically.
- ✓ RAG's price is latency on every single query plus an embedding-and-indexing pipeline you have to keep in sync forever.
- ✓ Fine-tuning updates weights from input–output pairs, so behaviour is baked in, inference stays fast, and no vector database is needed.
- ✓ Fine-tuning's price is thousands of curated examples, GPU time, a full retraining round for every update, and the risk of catastrophic forgetting.
- ✓ Prompt engineering steers attention toward patterns already in the weights: instant and free, and structurally incapable of adding knowledge.
- ✓ Sequence the work cheapest-first — prompt, then retrieval, then fine-tuning — because each stage produces the evaluation the next one needs.

What it covers

- 01 Why the same question gets different answers from different models** 0:00
Everything a model knows sits in its weights, frozen when training stopped. Every improvement technique is a way of working around that one fact.
- 02 RAG: retrieval, augmentation, generation** 1:29
The acronym is the architecture. Find relevant text, paste it into the prompt, then answer from it — with the model itself never modified.
- 03 Embeddings: finding documents that share no words with the query** 2:53
Both the question and every document become vectors, so relevance is measured as closeness in space rather than overlap in vocabulary.
- 04 What RAG costs: latency per query, infrastructure forever** 4:15
Retrieval is not free. It adds a round trip to every single request and a pipeline you have to operate indefinitely.
- 05 Fine-tuning: continuing training on a narrow dataset** 5:01
Take a model that already has broad competence and keep training it on examples of the behaviour you want.
- 06 How the weights actually move** 6:26

Supervised fine-tuning is a loss on predicted tokens, backpropagated into the weights, repeated across the dataset.

07 **The bill for fine-tuning: data, GPUs, maintenance and forgetting** 7:54

Four costs — and the fourth, losing general ability while gaining specialised ability, is the one that catches teams out.

08 **Prompt engineering: aiming attention at capability that is already there** 8:37

A prompt is not a request so much as a set of conditions. Change it and you change which learned patterns dominate the output.

09 **Where prompt engineering stops** 10:50

No phrasing adds information the weights never held, and finding a good prompt is empirical rather than derivable.

10 **Combining all three** 11:33

In production these are layers rather than alternatives, each covering a gap the other two leave open.

Glossary

Parametric knowledge — Information encoded implicitly in a model's weights during training. It cannot be listed, edited or deleted directly — only probed by asking the model.

RAG (Retrieval Augmented Generation) — Searching a corpus for passages relevant to a query, inserting them into the prompt, and generating an answer from them. The model itself is never modified.

Cosine similarity — The cosine of the angle between two vectors, used to score relevance. It ignores magnitude, so a long document and a short phrase about the same thing still match.

Chunking — Splitting documents into passages before embedding them. Chunk size and boundaries strongly affect whether retrieval can find anything.

Fine-tuning — Continuing training on a pre-trained model with a focused dataset, so the desired behaviour ends up stored in the weights.

Backpropagation — The algorithm that works out how much each weight contributed to the error, so the optimiser knows which direction to move it.

Catastrophic forgetting — Loss of general capability during specialised training, as the weights drift away from the configuration that supported it.

Attention mechanism — The component in each transformer layer that weighs parts of the input against one another, deciding what influences the output most.

Context window — The maximum amount of text a model can attend to at once. It bounds how much retrieved material a RAG system can supply.

Knowledge cutoff — The date at which a model's training data ends. Nothing after it exists in the weights, no matter how significant.

Vector embedding — A fixed-length list of numbers representing a piece of text, arranged so that texts with similar meaning produce nearby vectors.

Vector database — A store built for fast nearest-neighbour search over embeddings. It is the piece of infrastructure RAG obliges you to run and keep in sync.

Grounding — Constraining a model to answer from supplied text rather than from memory, usually by instruction. It is what makes a retrieved answer checkable.

Supervised fine-tuning (SFT) — Fine-tuning on explicit input–output pairs, where each example shows a request and the response the model should have produced.

Cross-entropy loss — The standard training objective for language models: it is large when the model assigned low probability to the token that was actually correct.

LoRA / PEFT — Parameter-efficient fine-tuning: freeze the base model and train small adapter matrices instead. Cheaper to train, cheaper to store, and reversible.

Few-shot prompting — Including worked examples in the prompt to demonstrate the desired output rather than describing it.

Inference — Running a trained model to produce output, as opposed to training it. RAG adds work here; fine-tuning moves work out of it.